Name: Mohammad Abu Shariha
User ID: mma7020

Problem 1: Lexical Analysis

1. Token, Pattern, Lexeme, and Symbol Table

a. A lexeme is a sequence of characters in the source program that is grouped together as a meaningful unit. Examples of lexemes are:

ip, <, and while from a code snippet like while (ip < z).

b. A token is a pair consisting of a name and an attribute, which serves as a label for a lexeme. For instance, ip is an identifier, so it might be represented as the token <ID, ip>. Tokens are what the lexical analyzer passes to the syntax analyzer (parser).

c. A pattern describes the set of all possible character strings that can form a lexeme for a particular token. These patterns are often defined using regular expressions. An example pattern for an identifier is a string of letters and digits that starts with a letter.

d. The symbol table is a data structure used to store information about identifiers, such as their name and other attributes. The lexical analyzer can optionally update the symbol table with all identifiers and numbers it finds.

2. Purpose of "Lookahead"

Lookahead is a technique where the lexical analyzer reads one or more characters beyond the current lexeme to decide where one token ends and the next one begins. This is necessary to resolve ambiguities in the input stream. For example, in a language that uses both = for assignment and == for equality, the lexical analyzer needs to look ahead after seeing the first = to determine if it should be an <ASSIGNMENT> token or part of an <EQUAL> token. Similarly, if a language has a keyword

if and also allows i as a variable name, lookahead is needed to differentiate between the two. Without lookahead, the lexer might incorrectly tokenize an input like: if(i==j).

3. Common Methods for Recovering from Lexical Errors

A lexical error occurs when the input cannot be recognized by any of the rules defined for the token classes. For example, a character like:

@ might be rejected as a lexical error in an identifier in Java. Common recovery methods include:

Panic Mode: This method involves deleting successive characters from the input stream until a valid token is found.

Deletion of a Single Character: One character is deleted from the remaining input to see if a valid token can be formed.

Insertion of a Missing Character: A character is inserted into the input to make it a valid lexeme.

Replacement or Transposition: An incorrect character is replaced with a correct one, or adjacent characters are swapped to form a valid token.

Problem 2

1. Transition Table in DFA vs. NFA

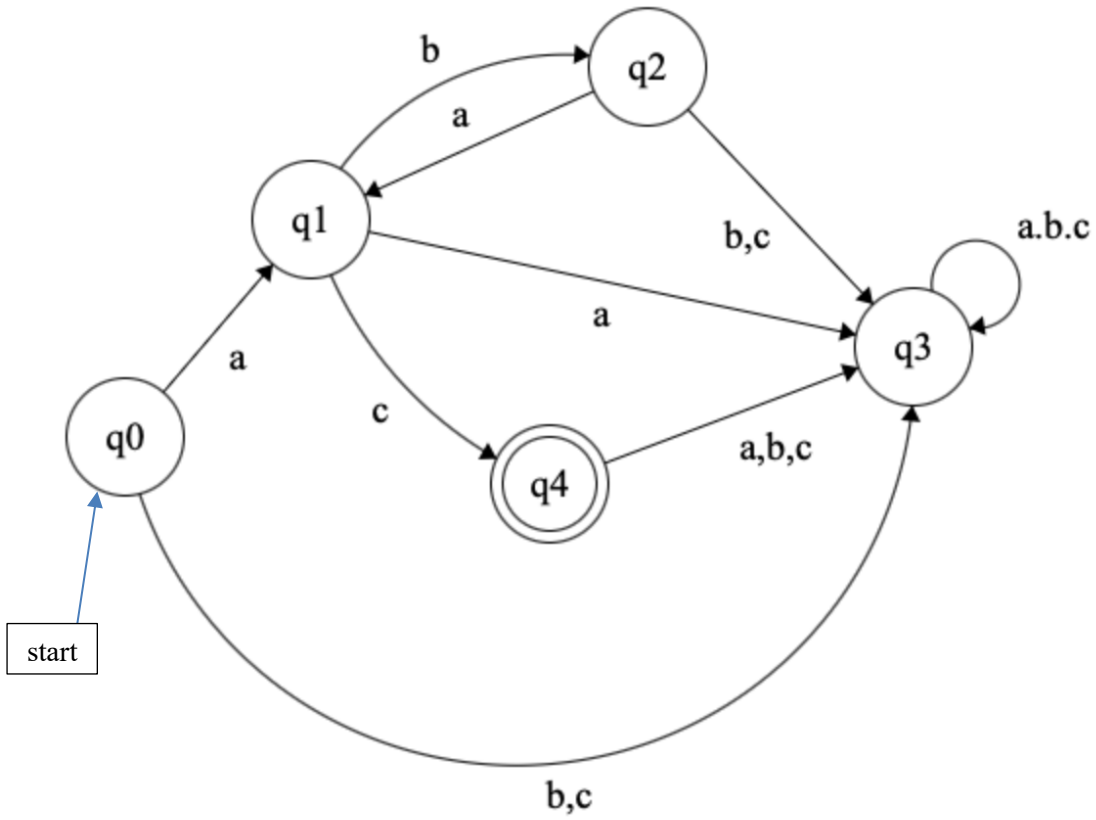
In a Deterministic Finite Automata (DFA), the transition table defines a function that maps a unique (state, input symbol) pair to exactly one next state. DFAs do not allow transitions on the empty string (€)

In a Non-Deterministic Finite Automata (NFA), the transition table can map a (state, input symbol) pair to a set of possible next states. An NFA can also have ϵ transitions, allowing it to change states without consuming any input. The key difference is the deterministic nature of the DFA, which means for any given input, there's only a single path the automaton can take, unlike an NFA.

2. DFA for (ab)∗ac

Transition table:

State	a	b	c
Q0 (start)	Q1	Q3	Q3
Q1	Q3	Q2	Q4
Q2	Q1	Q3	Q3
Q3	Q3	Q3	Q3
Q4 (end)	Q3	Q3	Q3



Problem 3:

1: NFA Transition Table

State	**x**	**y**	**z**
Q0	{q0, q1}	{q2}	{q0}
Q1	{q1}	∅	∅
Q2	∅	{q2}	{q2,q0}

Start state: q0 **Accepting state(s): q2**

2: DFA Construction (Subset Construction Method)

DFA State	x	y	z
{q0}	{q0, q1}	{q2}	{q0}
{q0, q1}	{q0, q1}	{q2}	{q0}
{q2}(accept)	∅	{q2}	{q0, q2}
∅	∅	∅	∅
{q0, q2}(accept)	{q0, q1}	{q2}	{q0, q2}

Reachable DFA states: {q0}, {q0, q1}, {q2}, {q0, q2}, ∅.

Accepting DFA states are the subsets that include q2: {q2}, {q0, q2}.

DFA start state: {q0} DFA accepting state(s): {q2}, {q0, q2}.

Problem 4:

1. (ab|ac)∗xy|x∗

This regular expression defines a language that is the union of two sets of strings.

The first part, (ab|ac)*xy, matches strings that are a sequence of ab or ac repeated zero or more times, followed by xy.

The second part, x*, matches any string of zero or more x characters.

The set is infinite, since it has strings with every length and there is no upper limit. The first eight shortest strings are:

1. Length 0: ϵ (the empty string, from x repeated 0 times).
2. Length 1: x.
3. Length 2: xx and xy.
4. Length 3: xxx.
5. Length 4: xxxx, abxy, and acxy.

The first eight shortest strings are:

ϵ, x, xx, xy, xxx, xxxx, abxy, acxy.

2. (abcd)+x

This regular expression matches one or more occurrences of the literal string "abcd", followed by the character "x". The set is infinite. The first 8 shortest strings are:

1. abcdx
2. abcdabcdx
3. abcdabcdabcdx
4. abcdabcdabcdabcdx
5. abcdabcdabcdabcdabcdx
6. abcdabcdabcdabcdabcdabcdx
7. abcdabcdabcdabcdabcdabcdabcdx
8. abcdabcdabcdabcdabcdabcdabcdabcdx

Problem 5:

Constructing a Regular Expression

The regular expression is [a-z][a-z]([a-z]|[0-9][a-z][a-z]+)*.

[a-z][a-z]: This ensures the string starts with at least two lowercase letters. The bracket notation [a-z] represents a character class that matches any single lowercase letter from a to z.

(...)*: This group can be repeated zero or more times, accommodating the rest of the string.

[a-z]: Inside the group, it can match any lowercase letter, allowing for sequences of letters.

|: The alternation operator allows for either the preceding pattern or the following one to be matched.

[0-9][a-z][a-z]+: This part handles the digit rule:

[0-9]: Matches a single digit.

[a-z][a-z]+: The [a-z] matches one letter, and the [a-z]+ matches one or more letters, together ensuring at least two letters follow the digit.

Problem 6:

1. Date Format: MM-DD-YYYY

The expression is (0[1-9]|1[0-2])-(0[1-9]|[12][0-9]|3[01])-(202[5-9]|20[3-9][0-9]|2[1-9][0-9][0-9]).

- (0[1-9]|1[0-2]): This part matches the month (MM). It handles numbers from 01 to 09 and 10 to 12.
- -(...): The hyphen matches the literal separator.
- (0[1-9]|[12][0-9]|3[01]): This part matches the day (DD). It handles days from 01-09, 10-29, and 30-31.
- (202[5-9]|20[3-9][0-9]|2[1-9][0-9][0-9]): This matches the year (YYYY) from 2025 to 2999. It correctly handles the range by matching years 2025-2029, 2030-2099, and 2100-2999.

2. Email Addresses

The expression is [a-zA-Z0-9]+([.-][a-zA-Z0-9]+)*@[a-zA-Z0-9]+\.[a-zA-Z]{3,6}.

- [a-zA-Z0-9]+([.-][a-zA-Z0-9]+)*: Matches the local part of the email. It starts with one or more alphanumeric characters, and then allows for segments separated by a dot or dash. This structure prevents special characters from being at the start, end, or appearing consecutively.
- @: Matches the literal @ symbol.
- [a-zA-Z0-9]+: Matches the domain name with one or more alphanumeric characters.
- \.: Matches the literal dot (.) before the top-level domain.
- [a-zA-Z]{3,6}: Matches the top-level domain, requiring 3 to 6 letters.

3. Comment Format

The expression is //[TBD]:\s.+.

- //: Matches the literal // characters.
- [TBD]: Matches one of the allowed tags: T, B, or D.
- :: Matches the literal colon that must follow the tag.
- \s: Matches the single literal space required after the colon.
- .+: Matches the comment text, which consists of one or more of any character, ensuring the comment is not empty.

Problem 7: Regular Expression IV

The regular expression to extract the timestamp, packet size, and IP address from each log entry is:

^\[(2[0-3]|[01]\d):([0-5]\d):([0-5]\d)\]\s.*?(\d+)KB\sto\sserver\s(192\.168\.0\.(?:25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d))$

Explanation of the regex:

- ^: Asserts the start of the string, ensuring the match begins at the beginning of a log entry.
- \[(2[0-3]|[01]\d):([0-5]\d):([0-5]\d)\]: This part matches and captures the timestamp.
    - \[and \]: Match the literal square brackets.
    - (...): Three separate capturing groups for hours (00-23), minutes (00-59), and seconds (00-59).
- \s.*?: A non-greedy match for the characters and space between the timestamp and the packet size.
- (\d+)KB: The fourth capturing group for the packet size, matching one or more digits followed by the literal string "KB".
- \sto\sserver\s: Matches the literal string " to server ".
- (192\.168\.0\.(?:...)): The final and fifth capturing group for the entire IP address.
    - 192\.168\.0\.: Matches the literal private IP prefix. The dots are escaped to be treated as literal characters.
    - (?:25[0-5]|2[0-4]\d|1\d\d|[1-9]?\d): This non-capturing group validates that the last octet of the IP address is a number between 0 and 255.
- $: Asserts the end of the string, ensuring the entire line is matched.